

Greedy pair merge

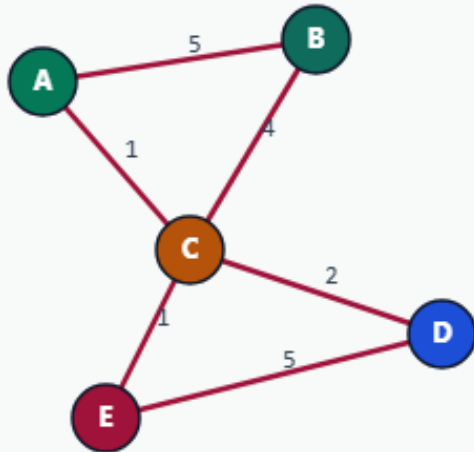
Greedy pair merge is the simplest coarsening engine that still feels like CAD

Every node starts alone

GREEDY PAIR MERGE

Every node starts alone

Step 1 / 5 · singletons · module01-03-greedy-pair-merge



Explanation

Agglomerative clustering begins with $K=5$ singletons. We repeatedly merge the highest-affinity legal pair until we hit target K .

- Target here: $K=2$
- No capacity limit in the reference run
- Priority = current edge-weight ranking

$K=5 \rightarrow$ aim for $K=2$
cutsize (all separate): 18

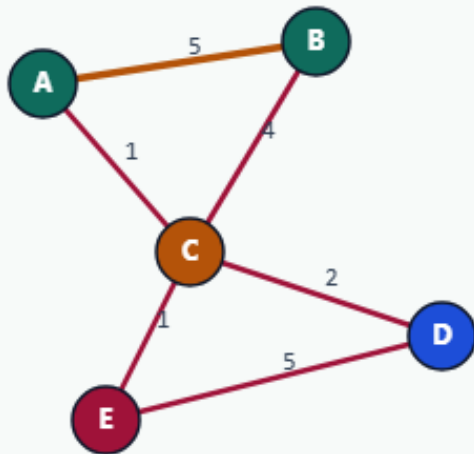
Every node starts alone

Merge 1: contract A-B

GREEDY PAIR MERGE

Merge 1: contract A-B

Step 2 / 5 · merge-ab · module01-03-greedy-pair-merge



Explanation

Heaviest edge is A-B at weight 5, so they merge first into a supernode. That protects the strongest connection immediately.

- Pick top legal pair on current graph
- Contract into a cluster
- Recompute edges to the new supernode

```
merge: A-B @ 5 → C0  
clusters: AB | C | D | E
```

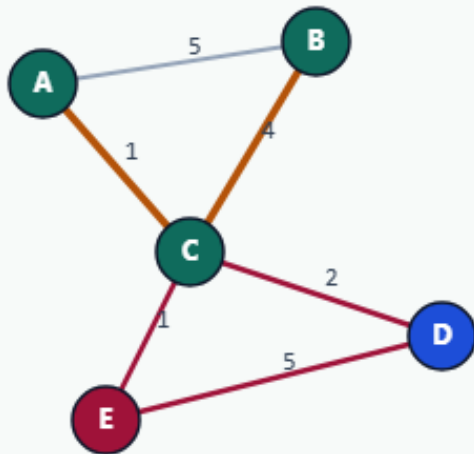
Merge 1: contract A-B

Merge 2: absorb C into AB

GREEDY PAIR MERGE

Merge 2: absorb C into AB

Step 3 / 5 · absorb-c · module01-03-greedy-pair-merge



Explanation

After contraction, C connects to {A,B} with combined weight 5. That ties D-E; alphabetical order merges C next, forming {A,B,C}.

- Supernode edges sum parallel connections
- C joins the A-B community
- Still two merges left toward K=2

```
merge: C-C0 @ 5  
clusters: ABC | D | E  
cutsize now: 8
```

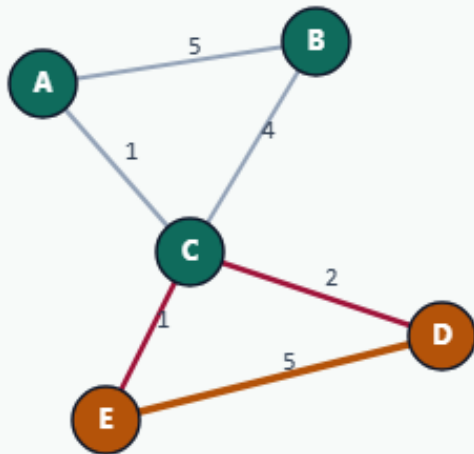
Merge 2: absorb C into AB

Merge 3: contract D-E

GREEDY PAIR MERGE

Merge 3: contract D-E

Step 4 / 5 · merge-de · module01-03-greedy-pair-merge



Explanation

Last merge joins D-E at weight 5. We now have exactly two clusters: {A,B,C} versus {D,E}.

- Reached target $K=2$
- Heavy edges sit inside clusters
- Only weak bridge edges remain cut

```
merge: D-E @ 5  
clusters: ABC | DE  
cutsizes: 3
```

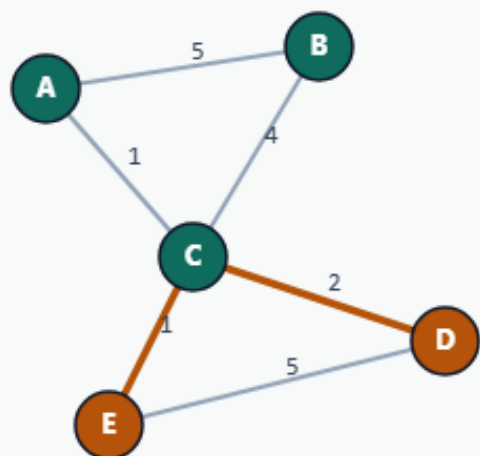
Merge 3: contract D-E

Result: cutsize 3

GREEDY PAIR MERGE

Result: cutsize 3

Step 5 / 5 · final-cut · module01-03-greedy-pair-merge



Explanation

Cut edges are C–D (2) and C–E (1). Greedy locked in the natural communities without backtracking—fast, but capacity rules can block this path.

- Golden: ABC|DE, cut=3
- Capacity=2 would forbid size-3 {A,B,C}
- Refinement labs repair worse seeds later

Final parts: ABC|DE
cutsize = 2+1 = 3

Result: cutsize 3

Browser lab track

- In the browser lab, load the starter, try the K and capacity presets, and run greedy merge
- Work the challenges

Implement track

- Run the reference greedy merge down to two clusters
- You should land on A-B-C versus D-E with cutsize three
- Re-implement contraction yourself and keep a deterministic tie-break so golden tests stay

Implement track — try these

```
# pwd — print working directory  
pwd
```

```
# ls examples — confirm tiny_graph.json is present  
ls examples
```

```
# greedy merge to K=2 via reference solver  
export PYTHONPATH=../common
```

```
python ../common/solvers.py examples/tiny_graph.json --k 2
```

Pitfalls to watch

- Greedy is myopic, a great first merge can strand you with a bad global cut
- Watch illegal merges that violate a size cap
- And updating affinities incorrectly after contraction creates silent wrong clusters until

Your turn

- Implement greedy pair merge end to end, hit a chosen K , and record cutsize
- Complete the checklist and quiz